

Mini-Project Big Data – SITCN2

Détection de Menaces de Cybersécurité en Temps Réel

Présentation du Projet

Ce mini-projet vous propose de construire un **système complet de détection de menaces de cybersécurité** selon l'**architecture Lambda**.

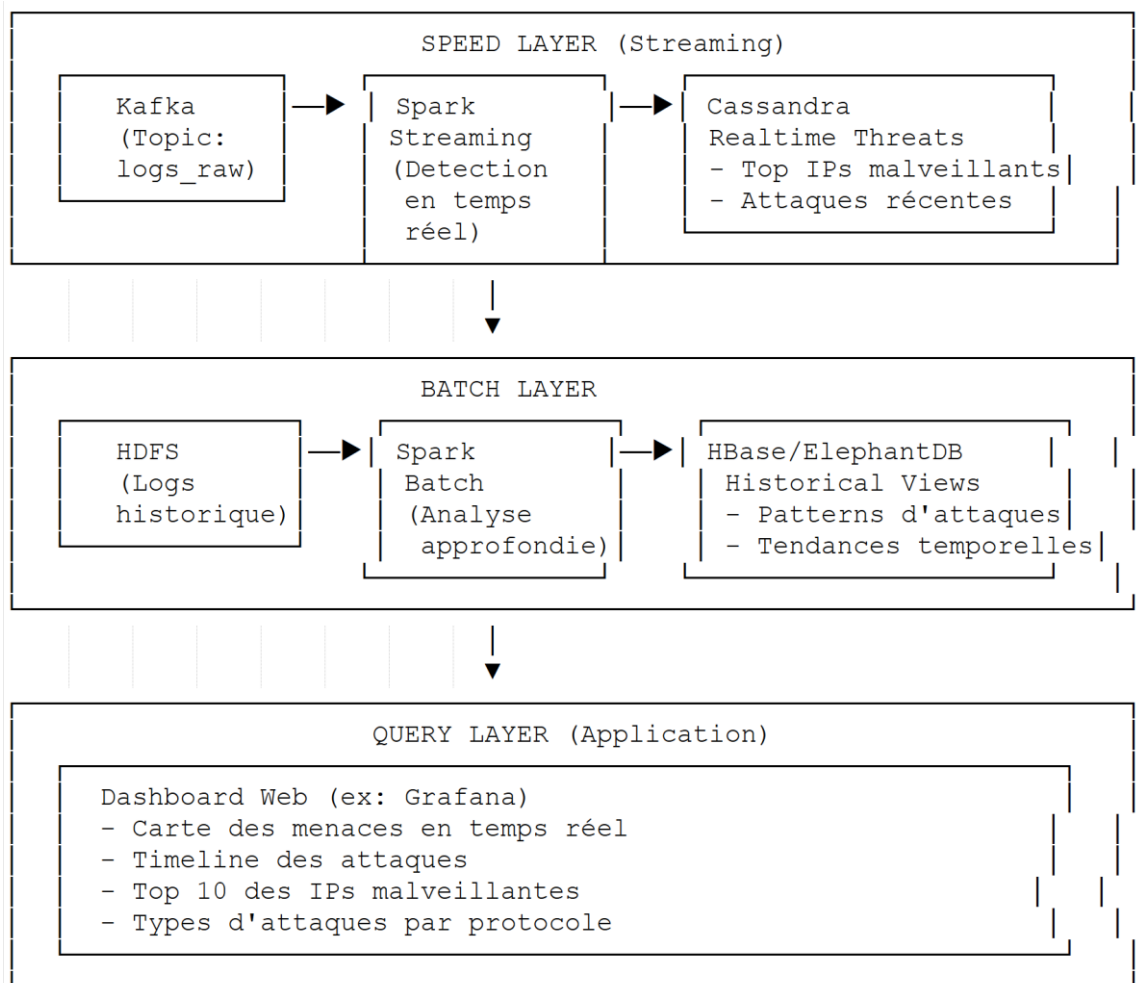
Vous analyserez un dataset de logs réseau (cybersecurity_threat_detection_logs.csv) disponible sur le lien suivant <https://www.kaggle.com/datasets/aryan208/cybersecurity-threat-detection-logs> pour :

- **Détecter des patterns d'attaques historiques** (analyse batch)
- **Identifier des menaces en temps réel** (streaming)
- **Visualiser les menaces actives et historiques** (dashboard)

Structure du Dataset

Champ	Description	Exemple
timestamp	Date/heure de l'événement	2023-10-15 14:23:45
source_ip	IP source de la connexion	192.168.1.45
dest_ip	IP destination	10.0.0.100
protocol	Protocole utilisé	HTTP, TCP, SSH
action	Action prise	allowed, blocked
threat_label	Classification de la menace	benign, suspicious, malicious
log_type	Type de système source	firewall, ids, application
bytes_transferred	Volume de données (bytes)	1542
user_agent	Client utilisé	Mozilla/5.0, sqlmap/1.7
request_path	Chemin accédé	/admin/login.php, /wp-admin/?id=1' OR '1'='1

Architecture Technique Proposée (Lambda Architecture)



Objectifs Techniques à Réaliser

A. Couche Batch

1. Stockage des logs dans HDFS

- Créer une structure de dossiers partitionnée par date (/logs/year=2023/month=10/day=15/)
- Utiliser `hdfs dfs -put` pour charger les données historiques

1. Analyse batch avec Spark

- **Top 10 des IPs sources malveillantes** (menaces suspicious + malicious)
- **Détection de scans de ports** : plusieurs connexions TCP vers des ports différents depuis la même IP en < 5 min
- **Analyse des chemins d'attaque** : extraction de patterns SQLi/XSS dans `request_path`
- **Volume de données transférées par type de menace** (corrélation `bytes_transferred` ↔ `threat_label`)

2. Stockage des vues batch

- Créer des tables HBase pour :
 - `ip_reputation` : Score de réputation par IP (basé sur historique)
 - `attack_patterns` : Patterns d'attaques fréquents
 - `threat_timeline` : Évolution temporelle des menaces

B. Couche Speed (TP3)

1. Pipeline Kafka

- Créer un topic cybersecurity-logs avec 3 partitions
- Simuler un flux continu de logs avec un producteur Kafka personnalisé
- 2. **Détection en temps réel avec Spark Streaming**
 - **Alerte immédiate** pour :
 - Tentatives de brute-force (5+ connexions échouées depuis même IP en 1 min)
 - Requêtes contenant des signatures d'attaques connues (sqlmap, nikto, ' OR 1=1--)
 - Volume anormal de données transférées (> 10 Mo en 10 sec)
 - **Score de menace en temps réel** par IP source
- 3. **Stockage des menaces actives**
 - Cassandra pour stocker les menaces détectées dans les dernières 24h
 - Schéma : ip_source → last_seen, threat_score, attack_types

C. Intégration Batch + Speed

- **Fusion des résultats** dans une API REST :
 - # Exemple de requête
 - GET /threats/ip/192.168.1.45
 - Retourne :
 - Score historique (batch layer)
 - Menaces actives (speed layer)
 - Recommandations de blocage

Livrables Attendus

Livrable	Description	Pondération
Code source	Projets Maven bien structurés (batch + streaming)	40%
Rapport	Architecture, choix techniques, schémas de données	20%
Dashboard	Visualisation des menaces (ex: Grafana ou simple HTML/JS)	25%
Démo	Présentation fonctionnelle du système complet	15%

Guide de Démarrage Pas à Pas

Étape 1 : Préparation de l'environnement

```
# 1. Démarrer le cluster Hadoop/Kafka
docker start hadoop-master hadoop-worker1 hadoop-worker2
docker exec -it hadoop-master bash
./start-hadoop.sh
./start-kafka-zookeeper.sh

# 2. Créer la structure HDFS pour les logs
hdfs dfs -mkdir -p
/data/cybersecurity/logs/{2023/10/15,2023/10/16,2023/10/17}
```

Étape 2 : Batch Layer (Analyse historique)

```
// Exemple : Détection de scans de ports avec Spark
Dataset<Row> logs =
spark.read().parquet("/data/cybersecurity/logs/*/*/*");

logs.filter("protocol = 'TCP'")
  .groupBy("source_ip", window($"timestamp", "5 minutes"))
  .agg(countDistinct("dest_ip.port").alias("distinct_ports"))
  .filter("distinct_ports > 20") // Plus de 20 ports différents
= scan probable
```

```
.write()  
.parquet("/data/cybersecurity/batch/port_scans");
```

Étape 3 : Speed Layer (Détection temps réel)

```
// Exemple : Détection de brute-force avec Spark Streaming + Kafka  
spark.readStream()  
  .format("kafka")  
  .option("kafka.bootstrap.servers", "localhost:9092")  
  .option("subscribe", "cybersecurity-logs")  
  .load()  
  .selectExpr("CAST(value AS STRING)")  
  .as(Encoders.STRING())  
  .filter(value -> value.contains("failed login"))  
  .groupBy(window($"timestamp", "1 minute"), $"source_ip")  
  .count()  
  .filter("count > 5") // Plus de 5 échecs en 1 min = brute-force  
  .writeStream()  
  .outputMode("update")  
  .format("console")  
  .start();
```

Étape 4 : Visualisation

- Utiliser **Grafana + InfluxDB** pour le dashboard temps réel
- Ou développer une simple page HTML qui interroge une API REST exposant les menaces

Idées d'Amélioration (Bonus)

1. **Machine Learning** : Entraîner un modèle de classification sur threat_label avec MLlib
2. **Corrélation d'événements** : Détecter des attaques multi-étapes (reconnaissance → exploitation)
3. **Géolocalisation** : Cartographier les origines géographiques des menaces
4. **Système de scoring adaptatif** : Ajuster le seuil de détection selon l'historique

Ressources Utiles

- [OWASP Top 10](#) : Signatures d'attaques à détecter
- [Sigma Rules](#) : Base de règles de détection
- [Apache Metron](#) : Référence d'architecture de cybersécurité Big Data
- TP1-TP2 -TP3

Critères de Réussite

- Architecture Lambda complète (batch + speed + serving)
- Détection de 3+ types de menaces différentes
- Latence < 5 secondes pour les alertes temps réel
- Visualisation claire des menaces historiques et actuelles
- Code bien structuré avec documentation minimale

Conseil pédagogique : Commencez par la couche batch (plus simple), puis ajoutez le streaming. Testez chaque composant isolément avant l'intégration complète.